Payment App API Documentation - Complete Reference

Base URL

/api/payments/pay/

Authentication

All endpoints require JWT authentication. Include the token in the Authorization header: Authorization: Bearer <your_jwt_token>

Allowed Roles: Regular User only (Company Owners cannot make payments)

Endpoint: Process Payment

POST /api/payments/pay/

Allowed Access: Authenticated Regular User only

Request Body:

json

```
{
  "booking_id": 55,
  "method": "fake_gateway"
}
```

Field Descriptions:

Field	Type	Required	Description
booking_id	integer	Yes	ID of the booking to pay for
method	string	Yes	Payment method ("fake_gateway" for testing)

Validation Rules:

- Booking must exist
- Booking must belong to authenticated user

- Booking status must be "pending_payment"
- Booking must not be expired (within 10 minutes of creation)
- Booking must have at least one passenger
- Booking must not already have a successful payment
- Method must be "fake_gateway" (for MVP)

Processing Flow:

1. Validate booking ownership and status
2. Check booking is not expired
3. Verify no duplicate payment exists
4. Check booking has passengers
5. Update booking status from "pending_payment" to "paid"
6. Create Payment record with status "success"
7. Generate unique tracking code automatically
8. Set amount from booking.total_price
9. Create Ticket records for each passenger
10. Return payment confirmation with ticket IDs

Successful Response

Status Code: 201 Created

json


```
{
  "payment": {
    "id": 12,
    "booking": 55,
    "amount": "900000.00",
    "tracking_code": "aB3dE5fG7hJ9kL1mN2pQ4rS6tU8vW0xYz...",
```

```
    "status": "success",
    "paid_at": "2026-05-30T10:15:32Z"
  },
  "ticket_ids": [98, 99]
}
```


Response Fields:

Field	Type	Description
payment.id	integer	Unique payment identifier
payment.booking	integer	ID of the paid booking
payment.amount	decimal	Amount paid (booking.total_price)
payment.tracking_code	string	Unique 120-character transaction code
payment.status	string	"success" for completed payment
payment.paid_at	datetime	ISO 8601 timestamp of payment
ticket_ids	array	List of generated ticket IDs for each passenger

---

Error Responses

400 Bad Request - Invalid Payment Method


```
json
{
  "method": ["Invalid payment method"]
}
```

400 Bad Request - Booking Already Paid

json

```
{
  "non_field_errors": ["Booking already paid"]
}
```

400 Bad Request - Booking Expired

json

```
{
  "non_field_errors": ["Booking has expired"]
}
```

400 Bad Request - Wrong Status

json

```
{
  "non_field_errors": ["Booking status isn't pending"]
}
```

400 Bad Request - No Passengers

json

```
{
  "non_field_errors": ["No passenger found for this booking"]
}
```

400 Bad Request - Wrong Owner

json

```
{
  "non_field_errors": ["You don't own this booking"]
}
```

401 Unauthorized - Missing/Invalid Token

json

```
{
  "detail": "Authentication credentials were not provided."
}
```

404 Not Found - Booking Doesn't Exist

json

```
{
  "detail": "Not found."
}
```

HTTP Status Codes Summary

Status Code	Meaning	When Used
201 Created	Success	Payment processed successfully
400 Bad Request	Client error	Validation failure
401 Unauthorized	Authentication required	Missing or invalid JWT token
403 Forbidden	Permission denied	User doesn't own the booking
404 Not Found	Resource not found	Booking ID doesn't exist

Payment Status Values

Status	Description	When Used
success	Payment completed successfully	After valid payment processing

Status	Description	When Used
pending	Payment in progress	(Future use for async payments)
failed	Payment failed	(Future use for failed gateways)

Booking Status Flow

text

Booking Created (pending_payment)

↓

Payment Processed (within 10 minutes)

↓

Booking Updated (paid)

↓

Payment Record Created (success)

↓

Tickets Generated

If payment not processed within 10 minutes: Booking status changes to "expired"

Tracking Code Information

- **Length:** 120 characters
- **Format:** Random alphanumeric string using get_random_string(120)
- **Uniqueness:** Enforced at database level
- **Generation:** Automatic on payment creation
- **Purpose:** Unique transaction identifier for reference

Important Business Rules

1. **One Payment Per Booking:** Bookings cannot be paid twice. Duplicate payment attempts return an error.
2. **Expiration Window:** Bookings expire 10 minutes after creation. Expired bookings cannot be paid.
3. **Ticket Generation:** Tickets are created automatically on successful payment. One ticket per passenger.
4. **Atomic Operation:** Payment creation, booking update, and ticket generation are wrapped in a database transaction. If any step fails, all changes are rolled back.
5. **Price Locking:** Payment amount is taken from booking.total_price at payment time, which is calculated as price_at_booking × number of passengers.
6. **Ownership Validation:** Users can only pay for bookings they own. Company owners cannot make payments.

Integration with Other Apps

Complete System Flow:

text

1. POST /api/bookings/

→ Create booking

→ Status: pending_payment

→ Returns booking_id: 55

2. POST /api/payments/pay/

→ booking_id: 55, method: "fake_gateway"

→ Process payment

→ Booking status: pending_payment → paid

→ Payment record created

→ Tickets generated

3. GET /api/bookings/my/

→ View paid booking

4. GET /api/tickets/ (if endpoint exists)

→ View generated tickets

Relationships:

- **Payment → Booking:** ForeignKey relationship
- **Ticket → Booking:** ForeignKey relationship
- **Ticket → Passenger:** ForeignKey relationship

Testing the Complete Payment Flow

Step 1: Create a Booking (Regular User)

bash

```
curl -X POST http://localhost:8000/api/bookings/ \
 -H "Authorization: Bearer <your_jwt_token>" \
 -H "Content-Type: application/json" \
 -d '{
  "trip": 12,
  "seat_ids": [1, 3],
  "passengers": [
   {
    "first_name": "Ali",
    "last_name": "Ahmadi",
    "national_code": "1234567890",
    "phone": "09123456789"
```

```
    }
  ]
}'
```

Response: {"id": 55, "status": "pending_payment", "expires_at": "2026-05-30T08:10:00Z", ...}

Step 2: Process Payment (within 10 minutes)

bash

```
curl -X POST http://localhost:8000/api/payments/pay/ \
  -H "Authorization: Bearer <your_jwt_token>" \
  -H "Content-Type: application/json" \
  -d '{"booking_id": 55, "method": "fake_gateway"}'
```

Response:

json


```
{
  "payment": {
    "id": 12,
    "booking": 55,
    "amount": "900000.00",
    "tracking_code": "aB3dE5fG7hJ9kL1mN2pQ4rS6tU8vW0xYz...",
    "status": "success",
    "paid_at": "2026-05-30T10:15:32Z"
  },
  "ticket_ids": [98, 99]
}
```


Step 3: Verify Booking Status

bash

```
curl -X GET http://localhost:8000/api/bookings/55/ \
  -H "Authorization: Bearer <your_jwt_token>"
```

Response: {"id": 55, "status": "paid", ...}

Example cURL Commands

Successful Payment

bash

```
curl -X POST http://localhost:8000/api/payments/pay/ \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..." \
  -H "Content-Type: application/json" \
  -d '{
    "booking_id": 55,
    "method": "fake_gateway"
  }'
```

Invalid Booking ID

bash

```
curl -X POST http://localhost:8000/api/payments/pay/ \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..." \
  -H "Content-Type: application/json" \
  -d '{
    "booking_id": 999,
    "method": "fake_gateway"
  }'
```

Missing Authorization Header

bash

```
curl -X POST http://localhost:8000/api/payments/pay/ \
```

```
-H "Content-Type: application/json" \
-d '{
  "booking_id": 55,
  "method": "fake_gateway"
}'
```

Version Information

- **API Version:** 1.0
- **Last Updated:** May 2026
- **Authentication:** JWT (JSON Web Tokens)
- **Required Role:** User (Regular user)
- **Payment Gateway:** Fake gateway for MVP testing
- **Production Ready:** Yes (after adding real payment gateway)
